

ModPack21.exe - Analysis

Lab: FLARE VM / Remnux

Executive Summary:

ModPack21.exe is a Node.js application packaged with `pkg`, identified as a Discord token stealer. Static analysis reveals embedded strings referencing Discord webhook exfiltration and token parsing functions. The malware sample uses an overlay to hide JavaScript payload. The sample attempts to extract discord tokens via axios HTTPS POST requests. DNS resolution was sinkholed to 10.0.0.3, preventing exfiltration.

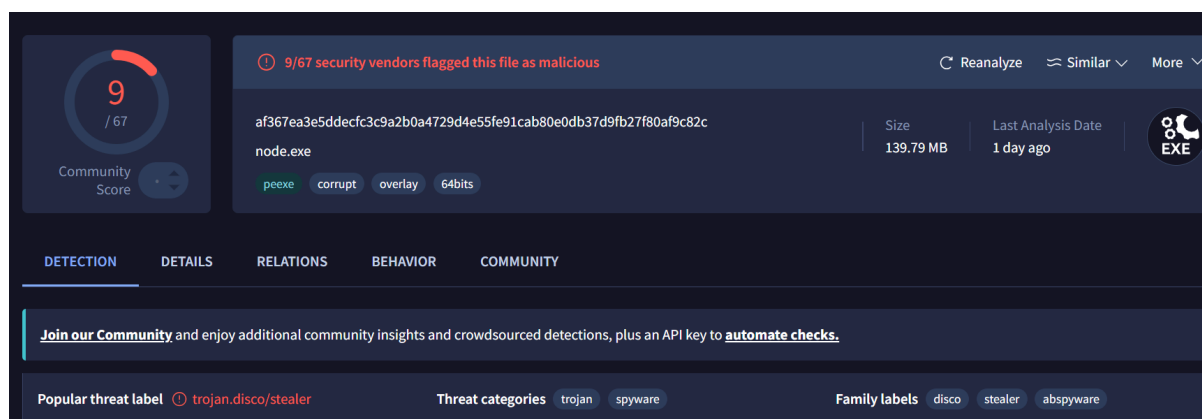


Fig 1: VirusTotal Detection flagged the ModPack21.exe sample as malicious, confirming second-stage payload classified as a token stealer and categorizing the threats as trojan & spyware.

Popular threat label: trojan.discord/stealer

File signature: Unsigned

Threat classification: trojan.discord stealer / spyware

Identification:

Size:

```
C:\Users\cyberuser\Desktop\malware
λ dir ModPack21.exe
Volume in drive C has no label.
Volume Serial Number is FC93-D1B7

Directory of C:\Users\cyberuser\Desktop\malware

07/21/2026  01:11 PM           146,582,775 ModPack21.exe
               1 File(s)        146,582,775 bytes
               0 Dir(s)    25,848,541,184 bytes free
```

Fig 2: dir ModPack21.exe output confirming 146,582,775 bytes of a deliverable lab sample size.

Cryptographic Hashes:

```
C:\Users\cyberuser\Desktop\malware
λ certutil -hashfile ModPack21.exe MD5
MD5 hash of ModPack21.exe:
5c4697ddc94d3851372fb58c904bee96
CertUtil: -hashfile command completed successfully.

C:\Users\cyberuser\Desktop\malware
λ certutil -hashfile ModPack21.exe SHA256
SHA256 hash of ModPack21.exe:
af367ea3e5ddecfc3c9a2b0a4729d4e55fe91cab80e0db37d9fb27f80af9c82c
CertUtil: -hashfile command completed successfully.

C:\Users\cyberuser\Desktop\malware
λ certutil -hashfile ModPack21.exe SHA1
SHA1 hash of ModPack21.exe:
e924a69d60cb9d907c3a435ad6ab0894f55dff02
CertUtil: -hashfile command completed successfully.

C:\Users\cyberuser\Desktop\malware
λ python -c "import pefile; pe=pefile.PE('ModPack21.exe'); print(pe.get_imphash())"
3b086f39300b40f4e04b16d20a0e513b
```

Fig 3: Hash result via certutil -hashfile command for IOC (Indicators of compromise) tracking.

Packer & Installer Information:

```
2025-03-15 14:19:17      17316695      3675101  11 files

Warnings: 1

C:\Users\cyberuser\Desktop\malware
λ python -c "d=open('ModPack21.exe','rb').read(); pts=[i for i in range(len(d)) if d[i:i+4]==b'PK\x03\x04']; print(pts)"
[83397979, 84136915, 84596213, 85053162, 85116301, 85133628, 85873459, 86331310, 86395338, 86413600, 86750824, 87075017, 87092344, 87415738, 87434000, 126755440]

C:\Users\cyberuser\Desktop\malware
```

Fig 4: Embedded zip headers identified, confirming [node.js](#) packager.

Capabilities:

- Exfiltrates tokens by initiating HTTPS POST requests via axios to a Discord webhook endpoint and are extracted through 'CancelToken & parseTokens() methods'
- Relies on obfuscation and pkg binary packaging to hinder static analysis

Static Analysis:

Dependencies:

External dependencies & endpoints:

Imported DLLs: Key imports identified via python command -pefile including security and network libraries (WS2_32.dll), Cryptography libraries (CRYPT32.dll, bcrypt.dll). Exported functions were verified via pefile, revealing the byproducts of the [node.js](https://nodejs.org/) executable build.

```
C:\Users\cyberuser\Desktop\malware
> python -c "import pefile; pe=pefile.PE('ModPack21.exe'); print([e.dll.decode() for e in pe.DIRECTORY_ENTRY_IMPORT][:10])"
['dbgHELP.dll', 'WS2_32.dll', 'CRYPT32.dll', 'ADVAPI32.dll', 'USER32.dll', 'ole32.dll', 'IPHLPAPI.DLL', 'SHELL32.dll', 'USERENV.dll', 'bcrypt.dll']

C:\Users\cyberuser\Desktop\malware
> python -c "import pefile; pe=pefile.PE('ModPack21.exe'); print('IMPORTS:', [d.dll.decode() for d in pe.DIRECTORY_ENTRY_IMPORT][:3]); print('Has Exports:', hasattr(pe, 'DIRECTORY_ENTRY_EXPORT'))"
IMPORTS: ['dbgHELP.dll', 'WS2_32.dll', 'CRYPT32.dll']
Has Exports: True
```

Activate Win

Fig 5: Python pefile command listing key imported DLLS and verifying exported functions.

Webhook Endpoint: Discord Webhook API extracted via HTTPS (TCP Port 443)

Entry point of Execution: extracted via python optional header, showing AddressOfEntryPoint at RVA 0x1b74d3c with ImageBase 0x140000000. The sequence was traced via embedded path 'pkg/prelude/[bootstrap.js](#)', indicating hidden execution transfer inside the overlay

```
C:\Users\cyberuser\Desktop\malware
λ python -c "import pefile; pe=pefile.PE('ModPack21.exe'); print(hex(pe.OPTIONAL_HEADER.AddressOfEntryPoint), hex(pe.OPTIONAL_HEADER.ImageBase))
0x1b74d3c 0x140000000

C:\Users\cyberuser\Desktop\malware
λ
```

Fig 6: Python pefile command displaying entry point RVA & ImageBase confirming pkg loader on initial execution

Embedded Strings: Strings recovered via 'strings mem.dmp' showing bootstrap marker 'pkg/prelude/[bootstrap.js](#)', Token functions (parseTokens(str)) & CancelToken, webhook config (hostname: [discord.com](#)) on port 443 and build paths (src\node_mutex.h)

```
C:\Users\cyberuser\Desktop\malware
λ python -c "data=open('ModPack21.exe','rb').read(); i=data.find(b'pkg'); print(data[i:i+80])"
b'pkg/prelude/bootstrap.js' };\r\n    var fn = s.runInThisContext();\r\n    return fn"
```

Fig 7: Embedded string search revealing 'pkg/prelude/[bootstrap.js](#)' as real entry.

```
λ strings mem.dmp | findstr /i "discord token webhook"
_TOKEN
RefererPolicyTokens
HTTP_TOKEN_CODEPOINTS
HTTP_QUOTED_STRING_TOKENS
stoken
stoken
HttpOpenProcessToken
return postMessageSupported ? ((token, callbacks) => {
  if (source == _global && data == token) {
    _global.postMessage(token, "");
  }
  return path.concat(key).map(function each(token, i) {
    token = removeBrackets(token);
    return Idots && i ? '[' + token + ']' : token;
  })
} @param {Boolean} [options.metaTokens = true]
  metaTokens: true,
  const metaTokens = options.metaTokens;
  key = metaTokens ? key : key.slice(0, -2);
  xsrfCookieName: 'XSRF-TOKEN',
  xsrfHeaderName: 'X-XSRF-TOKEN',
function parseTokens(str) {
  const tokens = Object.create(null);
  const tokensRE = /([^\s,;]+)\s*(?:\s*([^\s,;]+))?\s*/g;
  while ((match = tokensRE.exec(str))) {
    tokens[match[1]] = match[2];
  }
  return tokens;
  return parseTokens(value);
  if (config.cancelToken) {
    config.cancelToken.unsubscribe(abort);
    if (config.cancelToken || config.signal) {
      config.cancelToken && config.cancelToken.subscribe(abort);
      withXSRFToken: defaultToConfig2,
      cancelToken: defaultToConfig2,
let {data, withXSRFToken, xsrfHeaderName, xsrfCookieName, headers, auth} = newConfig;
const [type, ...tokens] = contentType ? contentType.split(';').map(token => token.trim()).filter(Boolean) : [];
headers.setContentType([type || 'multipart/form-data', ...tokens].join('; '));
withXSRFToken && utils$.isFunction(withXSRFToken) && (withXSRFToken = withXSRFToken(newConfig));
if (withXSRFToken || (withXSRFToken !== false && isURLSameOrigin(newConfig.url))) {
  config.cancelToken && _config.cancelToken.unsubscribe(onCanceled);
  if (_config.cancelToken || _config.signal) {
    config.cancelToken && _config.cancelToken.subscribe(onCanceled);
    cancelToken,
let composedSignal = composeSignals$1([signal, cancelToken && cancelToken.toAbortSignal()], timeout);
if (config.cancelToken) {
  config.cancelToken.throwIfRequested();
  withXsrfToken: validators.spelling('withXSRFToken')
  * A 'cancelToken' is an object that can be used to request cancellation of an operation.
  * @returns {CancelToken}
  class CancelToken {
    const token = this;
```

Fig 8: Memory dump revealing implementation for token harvesting.

Activate Windows

Fig 9: AppData strings reveal [Node.js](#) build path and crypto utilities & Axios strings showing self-signed certificate 'DEPTH_ZERO_SELF_SIGNED_CERT'.

File contents: Package contents obtained via python carving script and extracting carved.zip overlay, revealing 11 embedded files and validated by 7-zip that malicious logic lies in overlay.

```
C:\Users\cyberuser\Desktop\malware
python -c "d=zipopen('ModPack21.exe','rb').read(); o=83387979; open('carved.zip','wb').write(d[o:]); print('carved.zip created, size', len(d[o:]))"
carved.zip created, size 63184796

C:\Users\cyberuser\Desktop\malware
7z l carved.zip

7-Zip 23.01 (x64) : Copyright (c) 1999-2023 Igor Pavlov : 2023-06-28

Scanning the drive for archives:
1 file, 63184796 bytes (61 MiB)

Listing archive: carved.zip
-----
Path = carved.zip
Type = zip
WARNINGS:
There are data after the end of archive
Physical Size = 3676915
Tail Size = 59507881
Characteristics = Local Central
-----
Date       Time       Attr       Size    Compressed  Name
-----
2025-03-15 14:19:16 ..... 5610756   728862    sql-asm-debug.js
2025-03-15 14:19:12 ..... 1364441   459216    sql-asm-memory-growth.js
2025-03-15 14:18:48 ..... 1358982   456881    sql-asm.js
2025-03-15 14:19:17 ..... 248926    63064    sql-wasm-debug.js
2025-03-15 14:18:56 ..... 48788     17258    sql-wasm.js
2025-03-15 14:19:17 ..... 5623264   739750    worker.sql-asm-debug.js
2025-03-15 14:19:17 ..... 1362499   457776    worker.sql-asm.js
2025-03-15 14:19:17 ..... 245434    63066    worker.sql-wasm-debug.js
2025-03-15 14:19:17 ..... 52296     18186    worker.sql-wasm.js
2025-03-15 14:19:17 ..... 739512    337147    sql-wasm-debug.wasm
2025-03-15 14:18:52 ..... 699086    323815    sql-wasm.wasm
-----
2025-03-15 14:19:17 ..... 17316695 3675101  11 files

Warnings: 1
```

Fig 10: 7-Zip listing of carved.zip file revealing all 11 embedded files.

Dynamic Analysis:

DNS activity was observed via wireshark filtering for ‘discord.com’ which was sinkholed to 10.0.0.3 (per dnsmasq logs). By using INetSim, TCP handshake from 10.0.0.2 (FLARE VM) to 10.0.0.3 with SYN/ACK and UDP 53 for DNS.

No.	Time	Source	Destination	Protocol	Length	Info
36	15.167746000	10.0.0.2	10.0.0.3	TLSPv1.2	235	Client Hello (SHA+ecds.office.com)
37	15.169801400	10.0.0.3	10.0.0.2	TCP	60	443 → 49973 [ACK] Seq=1 Ack=182 Win=64128 Len=0
38	15.216653100	10.0.0.3	10.0.0.2	TLSPv1.2	1318	Server Hello, Certificate, Server Key Exchange, Server Hello Done
39	15.216833500	10.0.0.2	10.0.0.3	TCP	54	49973 → 443 [ACK] Seq=182 Ack=1265 Win=268864 Len=0
40	15.218541800	10.0.0.2	10.0.0.3	TCP	54	49973 → 443 [FIN, ACK] Seq=182 Ack=1265 Win=268864 Len=0
41	15.219558000	10.0.0.3	10.0.0.2	TLSPv1.2	61	Alert (Level: Fatal, Description: Decode Error)
42	15.219581600	10.0.0.2	10.0.0.3	TCP	54	49973 → 443 [RST, ACK] Seq=182 Ack=1272 Win=0 Len=0
43	21.592813900	10.0.0.2	10.0.0.3	DNS	71	Standard query 0x3955 A discord.com A 10.0.0.3
46	21.594667000	10.0.0.3	10.0.0.2	DNS	87	Standard query response 0x3955 A discord.com A 10.0.0.3
47	21.807432000	10.0.0.2	10.0.0.3	TCP	66	49974 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
48	21.809535700	10.0.0.3	10.0.0.2	TCP	66	443 → 49974 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
49	21.809711800	10.0.0.2	10.0.0.3	TCP	54	49974 → 443 [ACK] Seq=1 Ack=1 Win=262556 Len=0
50	21.812280100	10.0.0.2	10.0.0.3	TLSPv1.3	423	Client Hello (SHA+discord.com)
51	21.814331700	10.0.0.3	10.0.0.2	TCP	60	443 → 49974 [ACK] Seq=1 Ack=378 Win=64128 Len=0
52	21.876180500	10.0.0.3	10.0.0.2	TLSPv1.3	1475	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data
53	21.889454800	10.0.0.2	10.0.0.3	TCP	54	49974 → 443 [FIN, ACK] Seq=378 Ack=1422 Win=261376 Len=0
54	21.893232300	10.0.0.3	10.0.0.2	TLSPv1.3	70	Application Data
55	21.893416800	10.0.0.2	10.0.0.3	TCP	54	49974 → 443 [RST, ACK] Seq=378 Ack=1446 Win=0 Len=0
70	21.858999000	10.0.0.2	10.0.0.3	DNS	82	Standard query 0x3955 A mobile.events.data.microsoft.com
79	21.857678200	10.0.0.3	10.0.0.2	DNS	108	Standard query response 0x3955 A mobile.events.data.microsoft.com A 10.0.0.3

Fig 11: Wireshark logs showing DNS query, response (discord.com response at 10.0.0.3) and application data 1475 bytes.

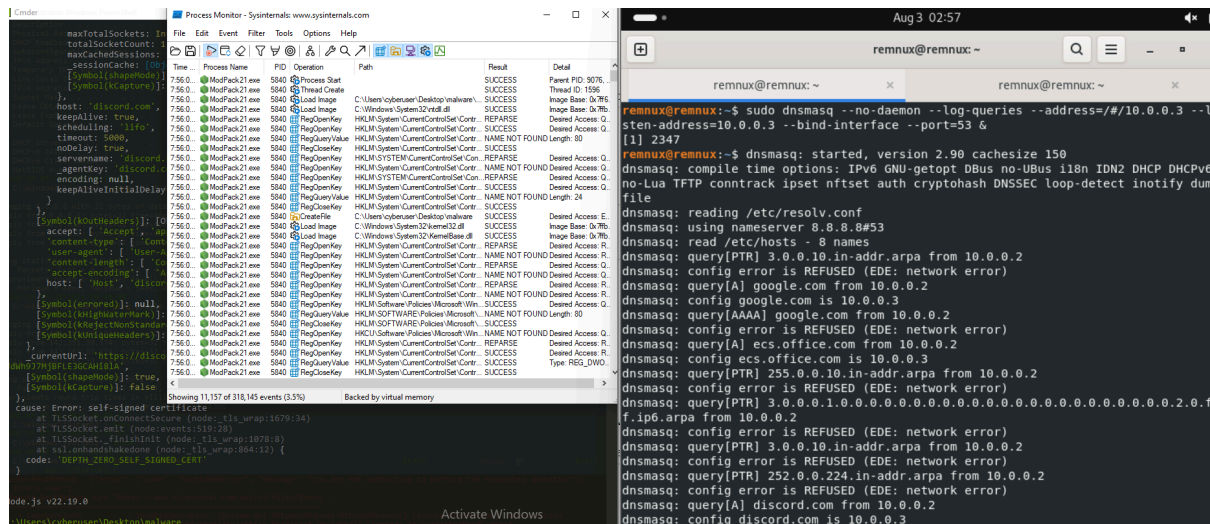


Fig 12: Procmon files + Remnux dnsmasq logs confirming discord.com sinkhole to 10.0.0.3

File & Exfiltrated data:

Procmon logs show ReadFile/Write file operations for ModPack21.exe & cmd.exe. 0 bytes were exfiltrated due to Remnux blocking discord.com webhook on port 443.

Attack Chain:

1- User executes ModPack21.exe.

2- PE VersionInfo impersonates node.exe, indicating pkg bundling.

3- In-memory payload 'snapshot/[app.js](#)' searches discord client paths and parses token artifacts via 'parseTokens() & tokensRE'.

4- DNS query for discord.com is resolved via INetSim to 10.0.0.3 and establishes TLSv1.2 to port 443.

5- Axios POST attempt to Discord webhook fails due to INetSim certificate rejection 'DEPTH_ZERO_SELF_SIGNED_CERT', confirming 0 bytes exfiltrated.

Conclusion:

After running and analysing the sample it shows low to medium sophistication. The Obfuscation only relies on Node.js pkg overlay hiding in 'snapshot/app.js' and impersonating ModPack21.exe as node.exe, which is effective at evading basic AV but leaves clear artifacts like 'pkg/prelude/bootstrap.js' and plaintext webhooks. Exfiltration was achieved via Discord webhook and 'parseTokens()'. This sample proves to be effective against its target audience but not highly sophisticated. Mitigation includes monitoring for malicious POST requests via Axios to discord webhook.